BUS RESERVATION AND MANAGEMENT SYSTEM

Programming Fundamentals Term Project Report

Group Members:

Mahadi Ur Rehman (2510382) – BS BIT

Arshan Mirza (2510376) – BS CMDA

Abdullah Javed (2510386) – BS BIT

1. Introduction

The **Bus Reservation and Management System** is an advanced, console-based C++ application designed to automate transport logistics, streamline passenger ticketing, and optimize fleet administrative workflows. The system introduces a robust, multi-layered architectural environment consisting of three dedicated portals: **User**, **Admin**, and **Finance**.

Passengers can securely register accounts, authenticate credentials, search schedules, reserve specific seats, and track their bookings via a systemic, token-based verification mechanism. The Admin panel facilitates comprehensive fleet management and emergency override controls (such as route decommissioning), while the newly integrated Finance panel generates critical corporate intelligence, including revenue analytics and refund auditing trackers. Persistent storage is managed across operational sessions through structured file handling routines.

2. Objectives of the Project

- Design and implement a secure, multi-role console application (User, Admin, and Finance) leveraging the C++ programming language.
- Enforce algorithmic **complex password validation rules** to guarantee user credential integrity.
- Engine dynamic **seat allocation maps** and unique alphanumeric token generation systems.
- Mitigate operational disruptions via automated contingency protocols handling **bus cancellations and refund routing**.
- Facilitate data-driven corporate decisions through specialized **financial reporting modules** (Revenue, Deficits, and Net Profit Margin analysis).
- Maximize the practical application of sequential file streams (`ifstream`/`ofstream`) to construct persistent transactional logs.

3. Problem Statement

Manual transit management frameworks are inherently susceptible to operational errors, such as overbooking, inefficient seat tracking, data redundancy, and a lack of granular audit logs. Furthermore, managing sudden route cancellations and processing individual passenger claims manually introduces significant human error and financial friction. This project addresses these critical operational vulnerabilities by introducing a computerized reservation infrastructure that dynamically synchronizes seating data, automates fallback refund queries, registers system-wide security logs, and generates structural financial summaries instantaneously.

4. Tools and Technologies Used

Tool / Technology	Purpose
C++ Language	Serves as the primary object-capable development platform.
<iostream>	Facilitates standard console input and output stream operations.
<fstream>	Executes persistent file handling, data insertion, and retrieval.
<string>	Enables robust textual data manipulation, formatting, and analysis.
<ctime>	Generates system timestamps for cryptographic security logging.
Text Files (.txt)	Serves as flat-file databases for operational records, finances, and security assets.

5. Programming Fundamentals Concepts Used

- **Structures (`struct`):** Employed to encapsulate cohesive data definitions, including `Bus`, `User`, and `Booking` profiles.
- **Static Arrays & Structural Tracking:** Utilized to map live seating arrangements, price metrics, and manage fixed boundary allocations (e.g., maximizing scales to 100 entries).
- **Sequential File Streams:** Extensive application of `ios::app` flag mechanics alongside temporary file shifting (`temp.txt`) to process dynamic deletion and runtime overwrites.
- **Sorting & Searching Frameworks:** Implementation of a targeted Bubble Sort mechanism to systematically arrange transport data based on Ticket Price, Route Name, or Bus Identification numbers.
- **String Parsing & Translation:** Algorithmic conversion routines, such as direct ASCII arithmetic reduction within `stringToInt()` and structural digit assembly in `intToString()`.
- **Input Validation Filters:** Custom boolean functions checking stream boundaries through character loops to isolate and reject non-conforming inputs, empty bounds, or whitespace injection.

6. Main Features of the System

- **Three-Tier Authentication:** Features independent security access protocols customized for Customers, Administrative Personnel, and Financial Auditors.
- **Multi-Criteria Data Sorting:** Empowers users to re-order transportation indices based on Pricing Tiers, Geographical Routes, or Fleet IDs.
- **Emergency Fleet Decommissioning Module:** Instantly purges a selected bus entry from standard availability files, transfers metadata archives to historical storage, purges outstanding passenger seat allocations, and generates pending refund records automatically.

- **Granular Input Filtering:** System-wide deployment of string inspection routines verifying exact parameter formatting for Bus IDs, Route names, and Time declarations (e.g., 08:00AM).
- **Real-Time Security Auditing:** Generates system logs continuously, chronicling strict time markers, active user identification, and operational operations executed.

7. System Modules

A. Authentication & Credential Management

- **Registration & Login Protocols:** Incorporates an account lockout mechanism that restricts system access upon reaching 3 consecutive invalid credentials.
- **Password Policy Guard (`MazbootPassword`):** Restricts account creation unless passwords meet rigorous security thresholds, including a minimum length of 8 characters, one uppercase character, a numerical digit, and a non-alphanumeric special character.
- **Profile Modification Utilities:** Provides secure operations for administrators to systematically adjust or remove customer login entries through targeted temporary file stream substitution.

B. Fleet & Ticketing Operations

- **Transit Asset Management:** Grants administrators the capability to register new vehicles, update route parameters, or erase specific transport models from active lists.
- **Interactive Ticket Allocation Engine:** Eliminates double-booking phenomena by strictly examining reservation tables prior to token generation and database assignment.

C. Financial Analysis & Auditing Controls

- **Revenue Optimization Reporting:** Computes individual bus asset returns, summarizes gross ticket income, and isolates peak operational assets.
- **Deficit Optimization Auditing:** Calculates comprehensive corporate financial losses emerging exclusively from verified and approved passenger refund applications.
- **Net Profit Estimator:** Computes clean corporate margins by executing standard balance sheet computations:

$$\textit{Net Profit} = \textit{Total Revenue} - \textit{Refund Loss}$$

8. Flat-File Database Architecture

To guarantee persistent states across runtimes, the system utilizes **9 specialized flat-file text databases**:

- `accounts.txt`: Restricts and houses verified user identity credentials.
- `buses.txt`: The master corporate repository containing active fleet specifications.
- `seats.txt`: A live database managing verified seat reservation data.

- `activity_log.txt`: The primary security register tracking system behaviors with corresponding time markers.
- `admin_log.txt`: Restricted access directory matching authorized administrative accounts.
- `reviews.txt`: A community feedback forum housing published customer evaluations.
- `destroyed_buses.txt`: An archive preserving information on decommissioned vehicles.
- `refunds.txt`: A financial tracking registry processing transactional refund claims (`Pending`, `Approved`, `Cancelled`).
- `finance.txt`: Private credential store managing financial officer login access.

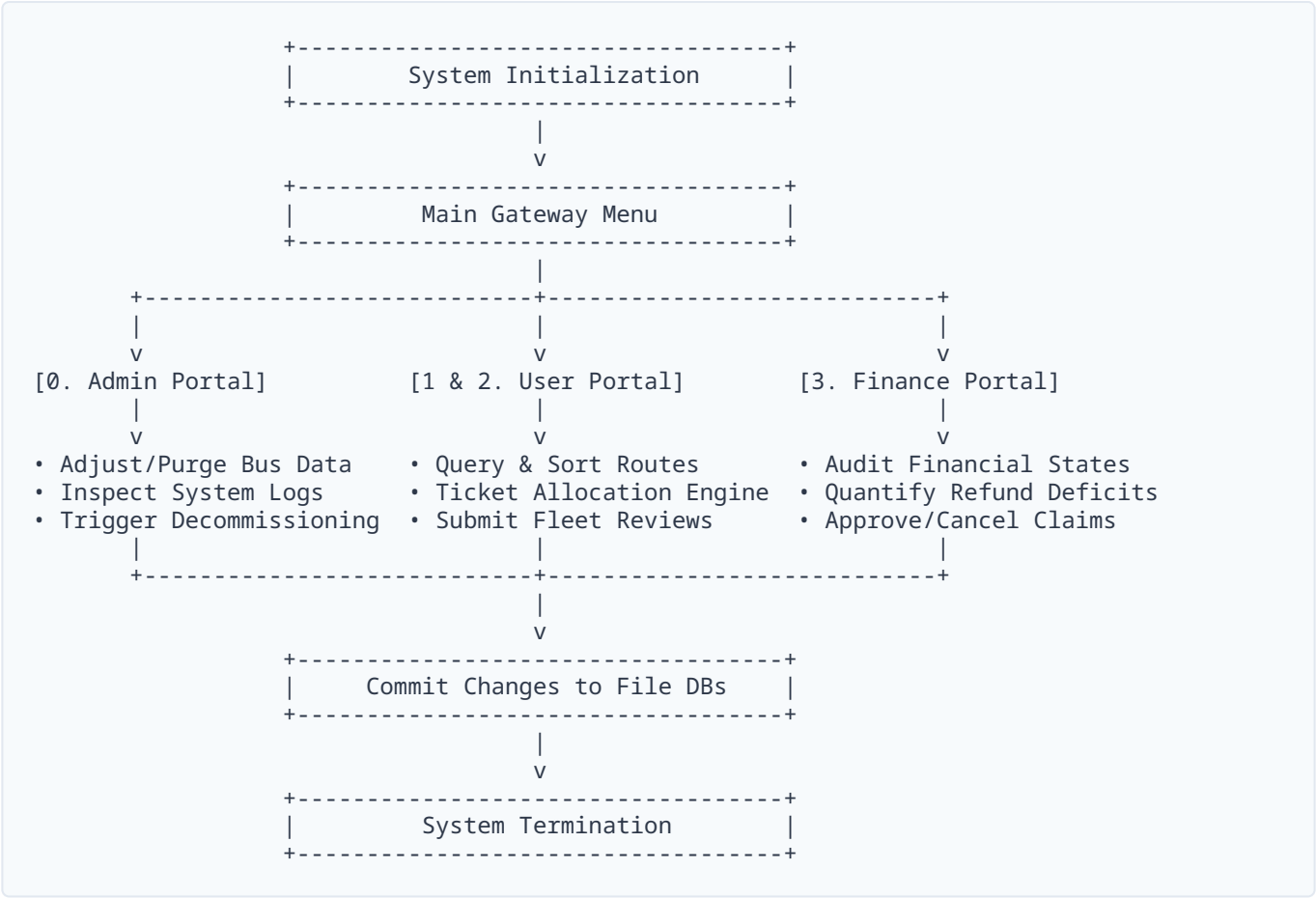
9. Security & Validation Protocols

- **Brute-Force Attack Mitigation:** Temporarily isolates user entry pathways upon registering continuous security check failures.
- **Cryptographic Token Generator:** Synthesizes custom validation strings by computing alphanumeric keys via specialized string formatting:

$$Token = f(BusNo, SeatNo, Username)$$

- **Data Stream Integrity Checks:** Routines such as `isAllDigits` and `isValidRoute` validate incoming inputs to prevent stream corruptions, variable overflows, or type-mismatch failures.

10. System Process Flowchart



11. Core Implementation & Algorithmic Breakdown

Manual Numerical Parsing Routine

To maximize computational visibility and maintain fundamental code paradigms without external transformation dependencies, translation from character arrays to basic integer values is handled manually:

```
int stringToInt(const string& str){
    int val = 0;
    for(int i = 0; i < str.length(); i++)
        val = val * 10 + (str[i] - '0'); // Sequential ASCII reduction logic
    return val;
}
```

Dynamic Array Sorting Engine

The application integrates a custom sorting algorithm optimized to systematically restructure complex array profiles based on real-time operational requests:

```
if(sortBy == 1){ // Organizes the array based on the lowest price tier
    int p1 = stringToInt(arr[j].price);
    int p2 = stringToInt(arr[j+1].price);
    if(p1 > p2) swapNeeded = true;
}
```

12. System Advantages

- **Automated Risk Management:** Completely eliminates the need to manually track individual passenger balances during unexpected transit cancellations.
- **Segregation of Duties:** Protects sensitive auditing registers by locking fiscal metrics within a specialized profile category, preventing standard operational layers from modifying financial tables.
- **Independent Execution Framework:** Runs seamlessly across alternative platforms due to its exclusive reliance on standard baseline libraries.

13. System Limitations

- **Console Design Interface:** Operationally constrained to character-based text inputs, lacking modern graphical controls.
- **Flat-File Constraints:** Lacks the multi-threaded query optimization found in traditional Relational Database Management Systems (RDBMS) such as SQL.
- **Static Allocation Bounds:** Constrained by pre-compiled buffer definitions, restricting concurrent processing memory to fixed limits of 100 entries per track.

14. Future Engineering Roadmap

- Migrate data management structures to an **SQL-backed database schema** to replace the sequential text file layout.
- Refactor the system architecture into a strictly object-oriented paradigm utilizing enterprise design patterns.
- Incorporate visual libraries (e.g., Qt or SFML frameworks) to deploy a highly interactive Graphical User Interface (GUI) dashboard.

15. Conclusion

The **Bus Reservation and Management System** represents an effective, programmatic framework designed to coordinate fleet allocation, customer verification, and fiscal tracking. By bridging modular operations with persistent flat-file mechanisms, the application demonstrates how traditional programming paradigms can be synthesized to resolve real-world logistical and operational challenges.